OBJECT-ORIENTED PROGRAMMING (OOP) ASSIGNMENT REPORT

Topic: Equipment and maintenance contract management system

Source Code Repository Link (GitHub): *OOP_Labw_06*

SECTION A: ANALYSIS AND DESIGN REPORT

A.1 Class and Interface Design

- **System components:**
  - `DeviceStatus`: `enum` type representing the operational and circulatory status of the device.
  - `Device`: Abstract `class` defining the common characteristics of devices and the interface for calculating maintenance costs.
  - `TeachingDevice`: Abstract `class` inheriting from `Device`, managing borrowing limits and state transition behaviours for borrowing/returning purposes.
  - `AirConditionerRemote`: Air conditioner remote device `class`.
  - `Microphone`: Microphone device `class`.
  - `ProjectorRemote`: Projector remote device `class`.
  - `ClassRoom`: Classroom `class` managing the collection of devices.
  - `IBorrower`: `interface` defining the borrower role.
  - `Lecturer`: Lecturer `class` implementing `IBorrower`.
  - `TeachingAssistant`: Teaching assistant `class` implementing `IBorrower`.
  - `BorrowItem`: `class` detailing a specific device item within a borrowing slip.
  - `BorrowingSlip`: Borrowing slip `class` managing borrow/return transactions.
  - Exception system: Base class `BorrowingException` and derived exception classes.
- **Relationships between components:**
  - **Inheritance:**
    - `TeachingDevice` inherits from the abstract class `Device`.
    - `AirConditionerRemote`, `Microphone`, `ProjectorRemote` inherit from `TeachingDevice`.

    - `DeviceUnavailableException`, `BorrowLimitExceededException`, `InvalidBorrowPeriodException`, `DuplicateDeviceException`, `DeviceNotInSlipException`, `DeviceAlreadyReturnedException` inherit from `BorrowingException`.
    - `BorrowingException` inherits from `System.Exception`.
- **Implementation:**
    - `Lecturer` and `TeachingAssistant` implement the `IBorrower` interface.
- **Aggregation:**
    - `ClassRoom` contains a `List<Device>`. Devices are assets equipped for the classroom but possess a lifecycle independent of the classroom (deleting the classroom does not destroy the devices).
- **Composition:**
    - `BorrowingSlip` is composed of `BorrowItem`s. The list of borrowing detail rows is directly owned by the borrowing slip and is destroyed when the borrowing slip is deleted.
- **Association:**
    - `BorrowingSlip` references `IBorrower`. The borrowing slip records the borrower's information; the borrower exists independently when the borrowing slip is deleted.
    - `BorrowItem` references `TeachingDevice`. The borrowing detail row records the handed-over device; the physical device exists independently when the borrowing slip or the borrowing row is deleted.

Detailed component specifications

1. Enumeration `DeviceStatus`

- **Responsibility:** Represents the operational and instantaneous circulatory statuses of devices within the system.
- **Values:**
    - `Available`: Device is ready for borrowing.
    - `Borrowed`: Device is currently borrowed for teaching purposes.
    - `UnderMaintenance`: Device is under maintenance / repair.
    - `Retired`: Device is retired / liquidated.

2. Abstract class `Device`

- **Responsibility:** Manages the foundational asset information of all devices; provides a method to calculate the projected annual maintenance cost via polymorphism.
- **Properties:**
  - `DeviceId: string` (`public get`) - Device identifier.
  - `DeviceName: string` (`public get` , `public set`) - Device name.
  - `YearOfUsage: int` (`public get` , `public set`) - Year put into usage.
  - `PurchasePrice: decimal` (`public get` , `public set`) - Initial purchase price of the device.
  - `Status: DeviceStatus` (`public get` , `protected set`) - Operational status.
- **Business Constraints (Invariants):**
  - `DeviceId` must not be empty or contain only whitespace.
  - `DeviceId` can only be set exactly once upon object instantiation.
  - `PurchasePrice > 0` .
  - `YearOfUsage <= DateTime.Now.Year` .
- **Constructors:**
  - `Device(string deviceId, string deviceName, int yearOfUsage, decimal purchasePrice, DeviceStatus status)` : Initialises foundational information and validates constraints.
- **Methods:**
  - `abstract CalculateAnnualMaintenanceCost(): decimal` : Abstract method to calculate the projected maintenance cost for one year.
  - `override ToString(): string` : Overrides the method representing device information as a string.
  - `MarkAsAvailable(): void` : Transitions status to `Available` (throws an exception if the device has been `Retired`).
  - `SendToMaintenance(): void` : Transitions status to `UnderMaintenance` .
  - `Retire(): void` : Transitions status to `Retired` .

3. Abstract class `TeachingDevice`

- **Responsibility:** Inherits from `Device` , appending properties and behaviours serving borrow/return operations in the classroom.
- **Additional properties:**
  - `MaxBorrowPeriods: int` (`public get` , `protected set`) - Maximum permitted borrowing periods.

    - `MaxBorrowDays: int` (`public get`, `protected set`) - Maximum permitted borrowing days.
- **Business Constraints (Invariants):**
    - Inherits all constraints of `Device`.
    - `MaxBorrowPeriods > 0`.
    - `MaxBorrowDays > 0`.
- **Constructors:**
    - `TeachingDevice(string deviceId, string deviceName, int yearOfUsage, decimal purchasePrice, DeviceStatus status, int maxBorrowPeriods, int maxBorrowDays)`: Forwards base parameters to `Device` and initialises borrowing limits.
- **Methods:**
    - `MarkAsBorrowed(): void`: Transitions status to `Borrowed` (only permitted when `Available`).

4. Derived classes of `TeachingDevice`

- `AirConditionerRemote` **class (Air conditioner remote):**
    - Inherits from `TeachingDevice`.
    - Overrides `CalculateAnnualMaintenanceCost(): decimal`.
        - Calculation method (student proposed): `PurchasePrice * 0.05m` (5% of the initial purchase price).
    - Overrides `ToString(): string`.
- `Microphone` **class (Microphone):**
    - Inherits from `TeachingDevice`.
    - Overrides `CalculateAnnualMaintenanceCost(): decimal`.
        - Calculation method (student proposed): `PurchasePrice * 0.10m` (10% of the initial purchase price).
    - Overrides `ToString(): string`.
- `ProjectorRemote` **class (Projector remote):**
    - Inherits from `TeachingDevice`.
    - Overrides `CalculateAnnualMaintenanceCost(): decimal`.
        - Calculation method (student proposed): `PurchasePrice * 0.05m` (5% of the initial purchase price).
    - Overrides `ToString(): string`.

5. `ClassRoom` class

- **Responsibility:** Manages the list of devices equipped in the classroom; calculates polymorphic maintenance costs and filters devices requiring maintenance.
- **Properties:**
  - `RoomId: string` (`public get`) - Classroom identifier.
  - `RoomName: string` (`public get` , `public set`) - Classroom name.
  - `Capacity: int` (`public get` , `public set`) - Capacity of the classroom.
  - `Devices: IReadOnlyList<Device>` (`public get`) - List of currently available devices in the room.
- **Business Constraints (Invariants):**
  - `RoomId` , `RoomName` must not be empty or contain only whitespace.
  - `Capacity > 0` .
  - Does not accept `null` device objects.
  - Cannot add two devices with the same code (`DeviceId`) into one room.
- **Constructors:**
  - `ClassRoom(string roomId, string roomName, int capacity)` : Initialises classroom information and initialises an empty device list.
- **Methods:**
  - `AddDevice(Device device): void` : Adds a device to the classroom after validation.
  - `RemoveDevice(string deviceId): bool` : Removes a device from the classroom based on the device identifier.
  - `FindDevice(string deviceId): Device?` : Searches for a device by its identifier.
  - `CalculateAnnualMaintenanceCost(): decimal` : Calculates the total projected maintenance cost of all devices in the room using polymorphism.
  - `GetDevicesRequiringMaintenance(): List<Device>` : Filters the list of devices currently in the `UnderMaintenance` status *or* having a usage time exceeding 5 years (`DateTime.Now.Year - YearOfUsage > 5`).

6. `IBorrower` interface and implementing classes

- `IBorrower` **interface:**
  - `BorrowerId: string` (`public get`) - Borrower identifier.
  - `FullName: string` (`public get`) - Full name of the borrower.
  - `GetMaximumDevices(): int` : Method returning the maximum number of devices permitted to be borrowed per slip.
- `Lecturer` **class:**
  - Implements `IBorrower` .

    - Constraint: Permitted to borrow a maximum of **3 devices** per slip (`GetMaximumDevices() => 3`).
- `TeachingAssistant` **class:**
    - Implements `IBorrower`.
    - Constraint: Permitted to borrow a maximum of **2 devices** per slip (`GetMaximumDevices() => 2`).

7. `BorrowItem` class

- **Responsibility:** Manages the details of a single borrowing instance for a specific device within the slip.
- **Properties:**
    - `Device: TeachingDevice` (`public get`) - The teaching device being borrowed.
    - `BorrowedDate: DateTime` (`public get`) - Borrowed date.
    - `DueDate: DateTime` (`public get`) - Due date to return the device.
    - `FromPeriod: int` (`public get`, `public set`) - Starting period of the borrow.
    - `ToPeriod: int` (`public get`, `public set`) - Ending period of the borrow.
    - `ReturnedDate: DateTime?` (`public get`, `private set`) - Actual returned date (default initialised to `null`).
    - `IsReturned: bool` (`public get`) - Returned status (`ReturnedDate != null`).
- **Business Constraints (Invariants):**
    - `Device` must not be `null`.
    - `DueDate >= BorrowedDate`.
    - `ToPeriod >= FromPeriod`.
    - `ReturnedDate` must initially be `null`.
    - Cannot return a `BorrowItem` twice (`IsReturned == true`).
    - Returned date cannot precede the borrowed date (`ReturnedDate >= BorrowedDate`).
- **Constructors:**
    - `BorrowItem(TeachingDevice device, DateTime borrowedDate, DateTime dueDate)`
- **Methods:**
    - `Return(DateTime returnedDate): void`: Executes the recording of the device return date.

8. `BorrowingSlip` class

- **Responsibility:** Manages the device borrowing slip and ensures the consistency of the borrow/return transaction.
- **Properties:**
  - `SlipId: string` (`public get`) - Borrowing slip identifier.
  - `Borrower: IBorrower` (`public get`) - Borrower.
  - `TeachingRoom: string` (`public get`) - Teaching room utilising the devices.
  - `BorrowedDate: DateTime` (`public get`) - Date the borrowing slip was created.
  - `Items: IReadOnlyList<BorrowItem>` (`public get`) - List of borrowed items within the slip.
- **Business Constraints (Invariants):**
  - `SlipId` , `Borrower` , and `TeachingRoom` must be valid (not `null` , not empty).
  - The borrower has not reached the device limit (`items.Count < Borrower.GetMaximumDevices()`).
  - The device does not already exist in the current borrowing slip.
  - The device must be in the `Available` status.
  - The borrowing days (`borrowDays`) must be greater than 0 and not exceed the maximum permitted days for the device (`MaxBorrowDays`).
- **Constructors:**
  - `BorrowingSlip(string slipId, IBorrower borrower, string teachingRoom, DateTime borrowedDate)`
- **Methods:**
  - `AddDevice(TeachingDevice device, int borrowDays): void` : Validates all business conditions; if valid, it then creates a `BorrowItem` , transitions the device status to `Borrowed` , and adds it to the `items` list. If an exception arises at any validation step, the device status and the `_items` list remain unaltered.
  - `ReturnDevice(string deviceId, DateTime returnedDate): decimal` : Searches for the borrowed item, verifies the return conditions, records the returned date, transitions the device status back to `Available` , and calculates the late penalty fee.
  - `CalculateTotalLateFee(): decimal` : Calculates the total late penalty fee for all devices in the slip.
    - Calculation method (student proposed): Iterates through the `Items` list, for each item where `ReturnedDate > DueDate` :

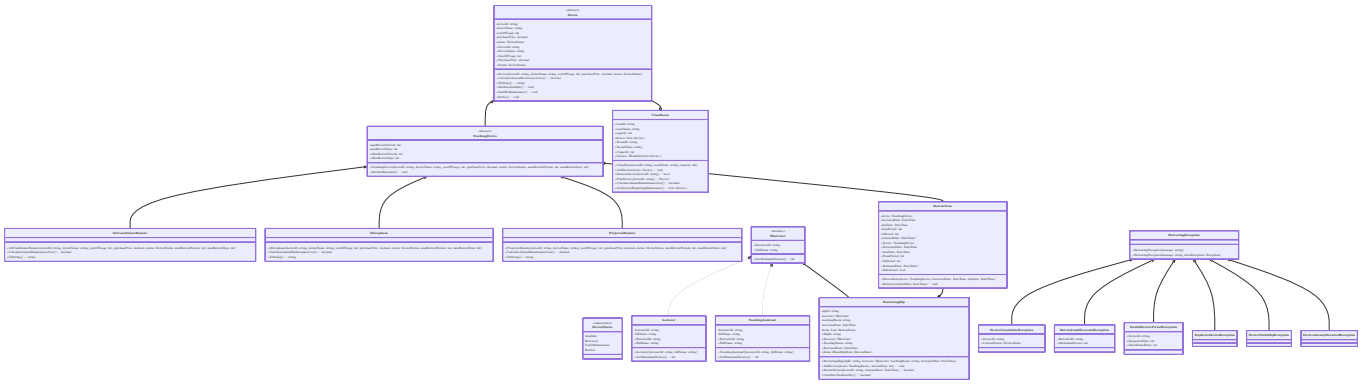
      $$\text{Penalty fee per item} = (\text{ReturnedDate.Value.Date} - \text{DueDate.Date}).\text{Days} \times 20,000$$

If returned on time or early (`ReturnedDate <= DueDate`), the penalty fee for that item is 0.

9. Business Exception Hierarchy

- **Base class:**
  - `BorrowingException` : Inherits from `System.Exception` . Provides two standard constructors: accepting `message` and accepting `(message, innerException)` .
- **Derived classes:**
  - `DeviceUnavailableException` : Raised when a device is not in the `Available` status. Contains `DeviceId: string` and `CurrentStatus: DeviceStatus` .
  - `BorrowLimitExceededException` : Raised when the borrower has reached the maximum device limit. Contains `BorrowerId: string` and `MaximumDevices: int` .
  - `InvalidBorrowPeriodException` : Raised when the number of borrowing days is not positive or exceeds the device's maximum days. Contains `DeviceId: string` , `RequestedDays: int` , and `MaxAllowedDays: int` .
  - `DuplicateDeviceException` : Raised when a device is added twice to the same borrowing slip.
  - `DeviceNotInSlipException` : Raised when requesting to return a device that is not within the borrowing slip.
  - `DeviceAlreadyReturnedException` : Raised when requesting to return a device that has already been recorded as returned previously.

A.2 UML Class Diagram



SECTION B: IMPLEMENTATION REPORT

The system is implemented using the `C#` language in accordance with the design in Section A and the assignment's requirements.

Implementation details:

B.1 Base class `Device` and statuses

- **Enumeration `DeviceStatus`:**
  - Declares 4 statuses: `Available`, `Borrowed`, `UnderMaintenance`, `Retired`.
- **Encapsulation and invariant protection of the `Device` class:**
  - Device identifier (`DeviceId`): Read-only property (`{ get; }`), assigned a unique value within the constructor, checks for non-emptiness via `string.IsNullOrWhiteSpace()`, throws `ArgumentException` if violated.
  - Properties `DeviceName`, `YearOfUsage`, `PurchasePrice`: Utilises `private` fields combined with condition checks within the `set` block. Throws `ArgumentException` if the name is empty; throws `ArgumentOutOfRangeException` if the purchase price is ≤ 0 or the year put into usage is greater than `DateTime.Now.Year`.
  - Property `Status`: Declares `{ get; protected set; }`. Provides status-altering methods including `SendToMaintenance()`, `MarkAsAvailable()`, and `Retire()`. Throws `InvalidOperationException` if intentionally transitioning the status whilst the device is in the `Borrowed` status.
  - Declares the abstract method `CalculateAnnualMaintenanceCost()` and overrides the `ToString()` method.

B.2 Specific device classes and Classroom Management

- **Abstract class `TeachingDevice`:**
  - Inherits from `Device`. Appends 2 properties `MaxBorrowPeriods` and `MaxBorrowDays`, checking the > 0 condition at the `set` block.
  - Implements the `MarkAsBorrowed()` method: Throws `InvalidOperationException` if `Status != DeviceStatus.Available`; otherwise assigns `Status = DeviceStatus.Borrowed`.
- **Derived device classes:**
  - The classes `AirConditionerRemote`, `Microphone`, `ProjectorRemote` inherit from `TeachingDevice`, forwarding parameters to the parent class constructor via `: base(...)`.
  - Overrides `CalculateAnnualMaintenanceCost()`:
    - `AirConditionerRemote`: Calculated as `PurchasePrice * 0.05m`.
    - `Microphone`: Calculated as `PurchasePrice * 0.10m`.
    - `ProjectorRemote`: Calculated as `PurchasePrice * 0.05m`.
  - Overrides `ToString()` to append the identification information of each device type.

- `ClassRoom` **class (Aggregation relationship):**
  - Manages the internal list using `private readonly List<Device> _devices`.
  - Exposes the list externally via the property `IReadOnlyList<Device> Devices => _devices.AsReadOnly()`.
  - `AddDevice(Device device)`: Throws `ArgumentNullException` if the device is `null`; throws `InvalidOperationException` if the `DeviceId` already exists within the room's list.
  - `RemoveDevice(string deviceId)` and `FindDevice(string deviceId)`: Throws `ArgumentException` if `deviceId` is empty or null. `RemoveDevice()` returns `true` if deletion is successful, returns `false` if not found.
  - `CalculateAnnualMaintenanceCost()`: Iterates through `_devices` using a loop and invokes the `device.CalculateAnnualMaintenanceCost()` method. Does not use `if` or `switch` statements to check the specific type of the device.
  - `GetDevicesRequiringMaintenance()`: Returns a list of devices satisfying `device.Status == DeviceStatus.UnderMaintenance` or `(DateTime.Now.Year - device.YearOfUsage) > 5`.

B.3 Borrow/return transaction processing and consistency assurance

- **Interface** `IBorrower` **and implementing classes:**
  - `IBorrower` declares the properties `BorrowerId`, `FullName` and the method `GetMaximumDevices()`.
  - `Lecturer`: Implements `IBorrower`, the `GetMaximumDevices()` method returns the value `3`.
  - `TeachingAssistant`: Implements `IBorrower`, the `GetMaximumDevices()` method returns the value `2`.
- `BorrowItem` **class (Composition relationship):**
  - Manages references to `TeachingDevice`, date fields `BorrowedDate`, `DueDate`, and borrowing periods `FromPeriod`, `ToPeriod`.
  - `ReturnedDate` has an initial value of `null`. The `IsReturned` property returns `ReturnedDate.HasValue`.
  - `Return(DateTime returnedDate)`: Throws `DeviceAlreadyReturnedException` if `IsReturned == true`; throws `ArgumentException` if `returnedDate.Date < BorrowedDate.Date`. Otherwise, assigns `ReturnedDate = returnedDate`.
- `BorrowingSlip` **class and transaction consistency assurance:**

- Manages the borrowing row list using `private readonly List<BorrowItem> _items`, exposed publicly via `IReadOnlyList<BorrowItem> Items`.
- `AddDevice(TeachingDevice device, int borrowDays)`: Verifies conditions in the following order:
  1. Device is `null` → throws `ArgumentNullException`.
  2. `borrowDays <= 0` or `borrowDays > device.MaxBorrowDays` → throws `InvalidBorrowPeriodException`.
  3. Device identifier already exists in the slip → throws `DuplicateDeviceException`.
  4. `_items.Count >= Borrower.GetMaximumDevices()` → throws `BorrowLimitExceededException`.
  5. `device.Status != DeviceStatus.Available` → throws `DeviceUnavailableException`.

  Only when passing all 5 conditions above does the system invoke `device.MarkAsBorrowed()` and add a new `BorrowItem` to `_items`. If an exception is raised at any validation step, the device status and the `_items` list remain unaltered.
- `ReturnDevice(string deviceId, DateTime returnedDate)`: Searches for the corresponding `BorrowItem` in the slip; if not found, throws `DeviceNotInSlipException`. If the item has been returned, throws `DeviceAlreadyReturnedException`. If valid, invokes `item.Return(returnedDate)`, invokes `device.MarkAsAvailable()`. Returns the late penalty fee calculated by the formula: `(returnedDate.Date - item.DueDate.Date).Days * 20000m` if returned after `DueDate`, otherwise returns `0m`.
- `CalculateTotalLateFee()`: Iterates through the entire `_items` list and accumulates the penalty fees of items having `ReturnedDate > DueDate`.

- **Business Exception Hierarchy:**
  - Base class `BorrowingException` inherits from `System.Exception`.
  - Implements 6 child exception classes inheriting from `BorrowingException`: `DeviceUnavailableException`, `BorrowLimitExceededException`, `InvalidBorrowPeriodException`, `DuplicateDeviceException`, `DeviceNotInSlipException`, `DeviceAlreadyReturnedException`.
  - The exceptions store error-raising contextual data via read-only properties (`DeviceId`, `BorrowerId`, `RequestedDays`, `MaxAllowedDays`, `CurrentStatus`).

SECTION C: TESTING REPORT

C.1 Initialisation and testing of standard business data

The system proceeds to load classroom data to test maintenance polymorphism, whilst simultaneously initialising teaching device borrow/return transactions according to the scenario requested by the assignment.

- **Device inventory data and classroom maintenance calculations (`CR-402`):**

Device ID	Device Name	Initial Usage Year	Purchase Price	Device Type	Prescribed Maintenance Rate	Projecte Maintena Cost
AC-402	Daikin Inverter Interface	2021	600,000	`ACRemote`	5% of purchase price	**30,000**
MIC-402	Shure Wireless Transceiver	2017	2,400,000	`Microphone`	10% of purchase price	**240,000**
PRJ-402	Sony Laser Projector Interface	2023	800,000	`PRJRemote`	5% of purchase price	**40,000**

- **Detailed formula for calculating classroom maintenance costs:**
  - **Total projected maintenance cost (Polymorphism):**

    $(600,000 \times 5\%) + (2,400,000 \times 10\%) + (800,000 \times 5\%) = 30,000 + 240,000 + 40,000 = \mathbf{310,000}$
  - **Periodic maintenance filtering check:** The device `MIC-402` was put into usage in 2017 ($\mathbf{2026 - 2017 = 9\ years > 5\ years}$), the system automatically filters it accurately into the list requiring periodic maintenance.
- **Borrowing slip transaction data (`SLIP-01` & `SLIP-02`) - Lecturer `Dr Alan Turing` (Limit of 3 devices):**

Slip ID	Teaching Room	Device ID	Borrowed Date	Due Date	Actual Return Date	Settlement Status	Accr Fine
SLIP-01	Lecture Theatre 1	**PRJ-01**	2026-03-10	2026-03-13	2026-03-13	Returned on schedule	**0**

Slip ID	Teaching Room	Device ID	Borrowed Date	Due Date	Actual Return Date	Settlement Status	Accr Fine
SLIP-01	Lecture Theatre 1	**MIC-01**	2026-03-10	2026-03-12	2026-03-14	Overdue by 2 days	**40,00**
SLIP-02	Seminar Room B	**AC-01**	2026-03-11	2026-03-15	*Unreturned*	In circulation	**0**

- **Detailed formula for calculating late penalty fees:**
  - **PRJ-01:** Return date coincides with the due date ($\textbf{2026-03-13} \leq \textbf{2026-03-13}$) $\Longrightarrow$ Penalty fee: 0.
  - **MIC-01:** Return date is 2 days later than the due date $\Longrightarrow$ Penalty fee: $(\text{2026-03-14} - \text{2026-03-12}) \times 20,000 = 2 \times 20,000 = \mathbf{40,000}$.
  - **Total penalty fee for slip `SLIP-01`:** $0 + 40,000 = \mathbf{40,000}$.
  - **Tracking ledger classification:** The system cleanly separates: The list of slips with late returned devices (`SLIP-01`) and the list of currently borrowed, unreturned slips (`SLIP-02`).
- **Testing results:**

```
================================================================================
  ASSET MANAGEMENT & LENDING SUBSYSTEM: VERIFICATION HARNESS
================================================================================
--------------------------------------------------------------------------------
  SUITE 1: FACILITY ASSET INVENTORY & MAINTENANCE VALUATION
--------------------------------------------------------------------------------
Classroom Initialised: 'Advanced Computing Laboratory' [ID: CR-402, Capacity: 45]
Registered Hardware: 3 unit(s) mounted.

Dynamic Maintenance Valuation: 310,000 (Expected: 310,000)
Servicing Audit: 1 unit(s) flagged for scheduled maintenance:
  - [MIC-402] Shure Wireless Transceiver (Tenure: 9 yr(s), Status: Available)

--------------------------------------------------------------------------------
  SUITE 2: NOMINAL LENDING, SETTLEMENT & LEDGER RECONCILIATION
--------------------------------------------------------------------------------
Registered Borrower: [Lecturer] Dr Alan Turing (ID: LEC-001) [Quota Limit: 3]
Provisioned Inventory: PRJ-01, AC-01, MIC-01 initialised in Available state.

Issued Slip 'SLIP-01' for Facility 'Lecture Theatre 1' on 2026-03-10.
Allocated Assets:
  - PRJ-01: Loaned for 3 day(s) | Due: 2026-03-13 | State: Borrowed
  - MIC-01: Loaned for 2 day(s) | Due: 2026-03-12 | State: Borrowed

Settled Asset [PRJ-01] on 2026-03-13 (Punctual). Fine: 0 | State: Available
Settled Asset [MIC-01] on 2026-03-14 (2 Days Overdue). Fine: 40,000 | State: Available

Ledger Audit Reconciliation:

[CATEGORY A: SETTLED LOANS WITH OVERDUE PENALTIES] (1 record(s))
  Slip ID: SLIP-01 | Borrower: Dr Alan Turing | Cumulative Penalty: 40,000
    -> [MIC-01] Shure Transceiver (Due: 2026-03-12, Returned: 2026-03-14, Latency: 2 day(s))

[CATEGORY B: ACTIVE OUTSTANDING LOANS] (1 record(s))
  Slip ID: SLIP-02 | Borrower: Dr Alan Turing | Facility: Seminar Room B
    -> [AC-01] Daikin Interface (Checkout: 2026-03-11, Due: 2026-03-15)
```

C.2 Boundary testing and exception testing

The testing programme in the `Test.cs` file verifies the execution of invariant rules, fault tolerance, and transactional integrity (Atomicity) strictly following the requirements in Section 11.2:

No.	Test Scenario	Mock Input Data	Expected Outcome
TC01	Borrowing a device under maintenance	`device.Status = UnderMaintenance`	Throws `DeviceUnavailableException`; admission to slip rejected.
TC02	Teaching assistant exceeding quota (> 2)	Teaching assistant adds 3rd device to the same slip	Throws `BorrowLimitExceededException`; item count remains 2.
TC03	Adding duplicate device to the same slip	Adding `MIC-DUP` a second time to slip	Throws `DuplicateDeviceException`; slip contents remain distinct.
TC04	Borrow duration exceeding device ceiling	Requesting 3-day loan (`MaxBorrowDays = 1`)	Throws `InvalidBorrowPeriodException`; device status remains `Available`.
TC05	Returning device not in current slip	Returning `PRJ-FOREIGN` not enrolled in slip	Throws `DeviceNotInSlipException`; return operation rejected.
TC06	Returning an already returned device	Invoking `ReturnDevice()` a second time for `PRJ-CLR`	Throws `DeviceAlreadyReturnedException`; redundant release prevented.
TC07	Return date precedes borrow date (atomicity)	Borrowed: `2026-04-10`, Returned: `2026-04-08`	Throws `ArgumentException`; device remains `Borrowed` without corruption.
TC08	Instantiating device with invalid price ($<= 0$)	`purchasePrice = -500000m`	Throws `ArgumentOutOfRangeException`.
TC09	Adding duplicate device	Adding duplicate `DeviceId` into `ClassRoom`	Throws `InvalidOperationException`; duplicate ID rejected within room.

No.	Test Scenario	Mock Input Data	Expected Outcome
	ID to the same classroom		
TC10	Loan period with end period < start period	`FromPeriod = 5`, `ToPeriod = 2`	Throws `ArgumentException`; timetable invariant preserved.

- **Exception testing results:**

